i4r-audit — team cheat sheet

v2.6.2

Set up once, then: reserve a paper → run the audit → file the result. Every dark window below was **captured from this tree on 2026-09-08**, not illustrated — so the counts are real and will drift. Check `cat VERSION`: this sheet describes **2.6.2**, and much of it changed in 2.3.0–2.4.x.

Read this first — the PATH trap that silently breaks the build

`pandoc`, `pdfinfo` and `pdftotext` live in the `i4rdoc` conda environment, not on your login PATH. In a plain shell here today `pandoc` is **missing** and `pdfinfo` resolves to miniconda's **21.03.0** — below the 21.06 floor, and it exits 99 on the `-custom` call the privacy gate makes. Nothing warns you: the build fails later, or R silently reports `pandoc_available() = FALSE`. **Put this at the top of every shell:**

EVERY TERMINAL, BEFORE ANYTHING ELSE

```
export PATH="$HOME/miniconda3/envs/i4rdoc/bin:$HOME/.local/bin:$PATH"
```

`pdfinfo -v` must print **21.06 or newer**. If it prints 21.03, your PATH is wrong.

1 What you need (one-time, ~30 minutes)

Thing	Where / how	Why — and what happens without it
Claude Code, on an account that can run Sonnet 5.5 (and Sonnet 5 for audits started before 2.7.0)	`npm install -g @anthropic-ai/claude-code`, then run `claude` and log in	Coordinator and the five auditors. Each audit keeps the model frozen in its snapshot: a new one spawns `claude-sonnet-5-5`
ChatGPT desktop app, updated (its Codex CLI 0.159.0 or newer), logged in, workspace with credits and GPT-6.1 Sol	chatgpt.com/download — the Codex CLI ships inside the app; nothing extra to install. The skill finds the newest working copy itself, wherever the app puts it (`python3 scripts/codex_cli.py --explain` shows its version)	Critics on GPT-6.1 Sol/high, adjudication on Sol/max (an audit started before 2.7.0 keeps the critic route it froze: GPT 5.6 Terra, or GPT-6 Sol for an audit started on 2.6.0). **No credits = no critique**; the critique model probe catches it, or a CLI or account without Sol, before spending. The probe runs on an audit's first round and after a CLI update, not per account: run every round of a Sol audit from an account that has Sol
jq	ships with macOS at `/usr/bin/jq`	**Hard.** Every journal append and all of finalize shell out to it. Never mentioned on the old sheet
Poppler ≥ 21.06 (`pdfinfo`, `pdftotext`)	in the `i4rdoc` env — see the PATH box above	**Hard.** Exact page count; the privacy gate calls `-custom` and `-meta`
pandoc	`i4rdoc` env; or set `PANDOC=<path>`	**Hard.** `build_docx.sh` searches three fallbacks; `run_verification.sh` does **not** — R must see pandoc itself
LibreOffice	libreoffice.org — a normal drag-to-Applications install is enough	**Hard, no degraded path:** `FATAL: LibreOffice/soffice is required`. Need not be on PATH — the app bundle is a fallback
R ≥ 4.4 with `knitr` + `rmarkdown`	`install.packages(c("knitr","rmarkdown"))`; add `readxl` / `haven` / `jmvReadWrite` as deposits need them	Only these two are unconditional; the rest follow the deposit's file types. `lavaan`, `psych`, `jsonlite` are **not used** — environment fingerprint only

Python 3	`pip3 install openpyxl`	The only third-party import in the skill, and it degrades rather than aborts. `jsonschema` is **not** required — the skill ships its own validator
The shared Dropbox discipline folder, synced locally	Ask the coordinator for the invite	Papers, packages, reservations, registry, completed reports

2 Install and prove it works

Terminal — install + self-tests

```
$ mkdir -p ~/.claude/skills && cd "$(mktemp -d)"
$ unzip -q "<discipline-root>/skill/i4r-audit-unified/i4r-audit.zip"
$ [ ! -e ~/.claude/skills/i4r-audit ] || mv ~/.claude/skills/i4r-audit ~/i4r-audit-old-$(date +%Y%m%d%H%M)
$ cp -R i4r-audit ~/.claude/skills/ && cd ~/.claude/skills/i4r-audit && chmod +x scripts/*.sh
$ cat VERSION   # the shared zip's version; then restart Claude Code
2.7.11
$ bash scripts/make_claims_script.sh --selftest
SELFTEST OK: 12 checks — R fallback, Stata-primary naming, author-inline, R-embedded, severity, tail
$ bash scripts/reserve_paper.sh --selftest
SELFTEST OK: 8 checks — claim, contention, idempotence, foreign release, conflict copy, completed guard,
stale flag
$ bash scripts/finalize_run.sh --self-test   # note the hyphen — see below
SELF-TEST PASSED
```

The same lines update an install. Every Mac holds its own copy, so a new shared zip reaches nobody until they run them again. `mkdir -p` matters on a first install: without `~/.claude/skills`, `cp -R` creates that folder from the skill's contents and Claude Code finds no skill. An old copy goes to your home folder, never beside the new one inside `skills/`, where Claude Code could load both.

The flag spelling is inconsistent, and one spelling lies to you

`make_claims_script.sh` and `reserve_paper.sh` take `--selftest` (no hyphen). Everything else — `finalize_run.sh`, `init_run.sh`, `log_event.sh`, `make_settled.sh`, `run_verification.sh` — takes `--self-test`. Get it wrong on `reserve_paper.sh` and you get a harmless usage line. Get it wrong on `make_claims_script.sh` and you get `FATAL: --self-test/comment.json not found — finalize the audit first`, which reads like a broken audit and is not.

The three `python3 -m unittest` suites must be run from the skill root (they use package-relative imports); every shell self-test works from anywhere by absolute path. Full battery, ~5 min:

FROM THE SKILL ROOT

```
python3 -m unittest discover -s tests -p 'test_*.py' # 493 tests
python3 -m unittest schemas.tests.test_v2_interfaces   # 36 tests
python3 config/check_policy_drift.py  ·  bash tests/test_run_critique.sh # ~12 min
```

The four iron rules

- **Never contact the paper's authors or the journal.** All correspondence goes through I4R — no exceptions, not even a “quick clarifying question.” The comment, filed with I4R, is the disclosure route. Personal data in a deposit never pauses or skips an audit: report it in the comment (identifier kind, file and column, counts, anonymisation claim, verified remedy) and never reproduce a value.
- **Never put a participant identifier beside a name, date, timestamp or clock time** in any audit artefact. Two separate gates exist to catch this; do not make them earn their keep.
- **Cannot verify a RETRACTION- or CORRIGENDUM-level claim after honest study? Return the paper** and take another. Never skip it; never let the AI decide for you.
- **Never name the discipline** in anything a replicator will see — every document is reused verbatim for the next cohort.

3 Reservations — automatic, but know what the board looks like

Six people work this tree at once. **The skill claims the paper for you the moment intake starts** and aborts cleanly if someone else holds it. All four intake refusals now run **before** the claim, so a rejected deposit leaves no reservation to unwind. Subcommands: `claim` · `release` · `status` · `list` · `reap` · `--selftest`. Always pass an **absolute** paper path — a relative one writes the claim inside the paper folder where the shared board cannot see it.

Terminal — the board, 2026-09-08 (real output, operator names replaced)

```
$ bash scripts/reserve_paper.sh list "$TREE"
  STALE  you@your-mac                11d  Journal_A_..._Title_of_paper_A01
         you@your-mac                 5d  Journal_B_..._Title_of_paper_A38
  ... 3 more, all STALE, all on other machines

$ bash scripts/reserve_paper.sh reap "$TREE"      # dry run — read-only
  WOULD REAP  11d claim (journal idle 11d): Title of paper A01
  OTHER     12d claim by them@their-mac: Title of paper A39
1 reapable, 4 held  (re-run with --force to release)
```

The printed lines never sum to the held count. Above, three `OTHER` lines are printed but **four** are held — the fourth is the 5-day claim, held silently because it is younger than the 7-day stale window. `reap` holds for six reasons and only five of them print: `CONFLICT COPY`, `UNREADABLE`, `OTHER` (someone else's), `UNKNOWN` (no audit findable), `ACTIVE` (journal still moving) — and silently, anything under the stale window. Cross-check against the summary line, never against the printed lines.

STALE is a pure age test, not "old and unfinished": a healthy audit on day 8 shows STALE. `reap` is owner-scoped — it never touches another machine's claim unless you set `I4R_REAP_ANY_OWNER=1`, and it deletes nothing without `--force`. Your claim is written at intake and released at publish, both automatic.

4 Run the audit

> **Unattended is the default since 2.3.0**
>
> A run goes intake → publish with **no authorization pause**. Set `I4R_AUTO_AUTHORIZE=0` to restore the attended stop. The grant is journaled and bound to the exact round plan either way; only the pause differs. INSTALL.md, SKILL.md and `run_critique.sh`'s header all say the same.

Open a terminal **at the tree root** (the folder holding `data/` and `runs/`, never the paper's own folder: Claude Code writes a `.claude/` folder where it starts, and nothing may be written into `data/`), export the PATH from the box on page 1, and start Claude Code. Type `/model`, pick **Sonnet 5.5**, effort **xhigh**. That choice governs *the coordinator you are talking to* only — the auditor, critic and adjudicator models are **policy-pinned** (`config/pipeline-policy-v2.json`, frozen into each audit's snapshot at init) and any other value is a hard `fail_early`. Then prompt through **`/goal`** rather than a plain message, phrased on the **deliverable** rather than the input path: long sessions compact their history, and "the audit is filed" stays checkable when a path reference no longer is.

PASTE INTO CLAUDE CODE — AFTER /MODEL → SONNET 5.5, EFFORT XHIGH

```
/goal Audit the paper in data/<Journal>/<Paper> end to end, starting with bash scripts/new_audit.sh
"data/<Journal>/<Paper>". Run the whole pipeline: workspace, five auditors, findings-registry
projection and adjudication, critique rounds to the policy target, verification notebook,
deliverable, claims script. Done means: finalize_run.sh reports the registry pair filed to the
DISCIPLINE-ROOT registry, and publish_completed.sh has filed the PDF + DOCX into
comments/<Journal>/<Paper>/ and the six record files into metadata/<Journal>/<Paper>/.
```

Write the four context files immediately after Phase 1, at the audit root: `PAPER.txt`, `ENV.txt`, `SLOW.txt`, `WRAPUP.txt`. `PAPER.txt` is checked in three places and `WRAPUP.txt` in one — so a missing `WRAPUP.txt` fails one step later than a missing `PAPER.txt`, and both fail *after* Phase 1 has already been paid for.

Two attempt knobs, easy to confuse: `MAX_TRIES` governs critique (and is *exported as* `I4R_MAX_ATTEMPTS`, overwriting it); `I4R_MAX_ATTEMPTS` governs Phase 1, the coordinator and the recovery scripts. Raising the wrong one does nothing.

5 The ordering trap before finalize

A registry/report edit or `invocation_correction` stales `SETTLED.json`, which the packet and receipt bind. Make those edits and gate-code edits **first**, then:

```
log_event.sh ×9 (8 phase_start/phase_end + 1 delivery) → approve → project (×2 if changed) →
make_settled.sh → log_event.sh settled used=true generated_after_round=1 entries=<n>
→ make_diff_packet.py --round N → run_post_delta_gates.py --round N → finalize_run.sh
```

`approve_settled.py` is the step people miss (SKILL.md names it, in the round-1 block): it binds the promotion approval to the current report hash, which `make_settled.sh` requires and never infers. Skip it and the settled cache promotes **zero** facts, silently. Skip the nine journal events and finalize still files, with `compliance_pass: false` and a NOTE — a green-looking finish on a permanently non-compliant record. Commit the nine in one transaction with `log_event.sh --batch <events.jsonl>`.

6 How long it actually takes

setup	five auditors	registry + adjudication	critique, all rounds	build + file
~5 min	7–35 min	1.8–13 h	0.5–4.0 h	20–40 min
environment recorded	parallel, one question each	every finding dispositioned	4 critics, then scoped	notebook, gates, publish

measured spreads across the three modern audits (n=3), from `run.json` phase records — not a target

A clean critique round is 10–20 minutes (critics run ~9–10 min in parallel; the rest is adjudication and integration). **A retry-heavy round 1 was observed at ~3.0 hours** — 16 critic invocations across four attempt waves. The policy target is **2 rounds, cap 7** (2.6.9; an audit initialised earlier keeps its frozen cap until `amend_round_cap.py` raises it with a journaled reason). The driver refuses a round above the audit's own cap, and any round on a finalized audit that was not reopened, before it touches the audit directory. Nothing loops the rounds for you — you or the session invoke each one. Of the audits finished on 2.6.x, most converged in 3–5 rounds; two stopped at the old cap with `cap_with_unresolved`.

Prefer invocation timestamps to phase stamps when you are timing anything: phase `start` has been logged as much as 15 minutes late.

7 Working the batch, and rescuing a killed run

Roughly 200 papers remain. Nothing else in the skill takes a tree root, so this is the only view of the whole board.

Terminal — read-only sweep, 2026-09-08

```
$ python3 scripts/status.py "$TREE" --stuck
21 audit(s); 8 need attention

aborted    idle  21.8d  r4  Title of paper A40
           phase1=None findings=None rounds_run=3 audit_id=20260101-audit-00000040
           ! unfinished and idle 21.8d (status=aborted)
           ! aborted

running    idle  12.6d  r1  Title of paper A41
           phase1=None findings=None rounds_run=None audit_id=20260101-audit-00000041
           ! unfinished and idle 12.6d (status=running)
```

`status.py` exits **1** when anything needs attention, in both text and `--json` mode, so a batch driver can branch on it. It finds audits by marker, so it sees the relocated workspaces under `.work/audits/` too. **One thing it does not tell you:** a held lock is only flagged on an unfinished, non-archived audit past `--idle-days` — 7 of 21 audits here hold a lock file and only 3 are flagged. A clean row does not mean no lock. It *does* now match each audit to its reservation and flags one that a finished audit never released (before 2.4.2 it matched none, because claims record an absolute path on the *claiming* machine).

A killed Phase 1 no longer bricks the audit. Sleep, OOM, reboot or a mistimed Ctrl-C strands an attempt and the guard then refuses to dispatch, correctly, to avoid a duplicate paid call:

Prevention beats recovery: run `caffeinate -i` in another tab for any overnight run. A sleeping Mac is the documented cause of every stranded attempt in this tree.

`renumber_findings.py` converts role-scoped ids (`REP-f1_d100…`, `TXT-TVD-F2`, `P1` / `TN1`) to the contiguous `F1..FN`

```
python3 scripts/run_phase1.py "$AUDIT_DIR" --
recover-stranded
```

A metadata-2.1 strand sits at state `provider_requested`, **not** `inference_started`. Recovery of a 2.1 strand records `attempt: null` and **charges nothing**, so your retry budget is untouched (INSTALL.md says the same). A pre-2.1 workspace cannot be recovered and says so.

the projection requires. It *reorders* as well as renames — numbering follows marker order in the report. Run it **once, right after `run_coordinator.py` and before round 1**: critic verdicts join by id, so renumbering after a round orphans that round's verdicts.

```
python3 scripts/renumber_findings.py "$AUDIT_DIR"
--dry-run
python3 scripts/renumber_findings.py "$AUDIT_DIR"
--apply --also report/ADJUDICATION.md
```

Renumbered late anyway? Integrate that round under its old ids first, then rerun with `--also` on each `critique/round_N/*_BEFORE_INTEGRATION.*` snapshot (INSTALL.md).

Do not run these four — they are pinned to an audit the driver has moved past

`recover_critique_probe.py` · `recover_round1_projection.py` · `retry_completeness_once.py` · `retry_targeted_adjudication_once.py`. A stranded operator grepping `scripts/` for "retry" will find them. They **fail in ways that read like a corrupt audit rather than a stale script.** Ask in the channel instead.

8 What lands where, and how you know it is done

```
<discipline-root>/
  data/<Journal>/<Paper>/ — article PDF +
package. Read-only.
  runs/<Journal>/<Paper>/audit/ — full working
record: reports,
  ledger, adjudication, critique rounds,
notebook, run.json
  comments/<Journal>/<Paper>/ — the comment:
.pdf + .docx
  metadata/<Journal>/<Paper>/ — 6 files:
claims_script.R ·
    comment.json · run.json ·
finalization_receipt.json ·
    document_build_manifest.json · release-
manifest.json
  registry/ — index.jsonl + one dir per audit
  reservations/ — your claim vanishes on publish
  old_runs/ — everything from before 2026-09-22,
and superseded publications
```

Eight files per publication, split since 2.6.2: two in comments/, six in metadata/. Start an audit with `bash scripts/new_audit.sh "data/<Journal>/<Paper>"`.

Three signs, all required. Publish prints the reservation release on the line *above* `published:` — that line is sign 3, and `REFUSED` or `note: no reservation to release` means it did not happen.

Terminal — verify a finished audit

```
$ python3 scripts/finalization_receipt.py check
"$AUDIT_DIR"
$ python3 -c "import json;print(json.load(
    open('$AUDIT_DIR/finalization_receipt.json'))
['registry']['root'])"
../../../../registry       # correct
(runs/<J>/<P>/audit)
../../../registry          # correct (legacy
<J>/<P>/audit)
../../../skill/i4r-audit-unified/registry   # WRONG
../../../../../../../.claude/skills/registry # WRONG
```

Check the registry path on every audit — two of three were mis-filed before 2.4.2

Finalize announces which registry it chose, but **read the path, not the label**: it must end in `<discipline-root>/registry`. If `REGISTRY_DIR` or `I4R_REGISTRY_DIR` is set in your shell, **unset it** — finalize refuses one that is not the tree's own registry (set `I4R_REGISTRY_OVERRIDE=1` only if filing elsewhere is intended). `publish_completed.sh` and `finalization_receipt.py` do **not** read those variables and take no `--registry`: they read the registry from `finalization_receipt.json`. Choose it once, with `finalize_run.sh --registry`. The `ok: filed into …/registry/…` line older docs told you to wait for was dead code and never printed; it was removed in 2.4.2. Verify with the command above instead.

9 When something goes wrong

You see	It means / do this
`FATAL: audit dir contains a character that disarms the filesystem sandbox`	The title holds `? * [ ] { } \` — roughly one paper folder in fourteen here. Those compile as globs and silently deny every sandboxed read while preflight still reports success. Put the audit dir somewhere clean and name the real folder with `I4R_PAPER_DIR`; the reservation is still claimed under the true name.
`FATAL: package archive is corrupt or truncated … zipinfo -t exits 3`	Structurally broken deposit, refused before the claim. Re-download. A **NOTE** about members that "do not inflate cleanly" is *not* this — the archive is fine and the audit proceeds.
`FATAL: the audit directory or the deposit holds Dropbox conflicted copies`	Delete them — they are never the canonical file. The scan now covers the deposit as well as the audit dir.
`unresolved Phase-1 invocation … (state provider_requested)`	A killed run. `run_phase1.py <dir> --recover-stranded`. If it instead says **still in flight (provider pid N is alive)**, another process holds the audit — wait, do not recover.
`[wait ] critic_x <class> failure; retrying in Ns`	Normal. `<class>` is only ever `transport`, `throttling`, `provider_5xx`, `timeout` (our own 45-min wall-clock kill) or `schema`. Anything permanent exits instead, with `[fail ] … permanent failure on attempt N`.
`Round never returns at all`	A pre-2.4.1 checkout: a timed-out call consumed no budget and the retry loop re-dispatched forever. `cat VERSION` — you need 2.4.1.
`[fail ] critic_x attempt N`	Read `critique/round_N/invocations/<id>/attempts/<role>/<attempt>/failure.log`; `stderr.txt`, `events.jsonl` and `validation.txt` sit beside it. Usually credits or network.
`[stop ] critic_x: snapshot integrity check failed before dispatch; no provider call was made`	The frozen snapshot's *content* changed, not just its mode. Nothing was spent. In the audit directory run the command the next line prints, `chmod -R u+w <dir> && rm -rf <dir>` for `critique/round_N/snapshots/<sha>` or `critique/round_N/target_snapshots/<sha>` (a plain `rm -rf` cannot remove a frozen tree), and rerun: it is content-addressed and rebuilt identically. `[stop ] … snapshot modes keep changing (content verified)` is different: nothing changed, but a write bit kept coming back after three re-freezes, so deleting cannot help. Stop the sync client or move the audit off the synced folder, then rerun. The same fault after a call reads `[fail ] … snapshot changed after the provider call (environment, not the critic); not retried`. By contrast `[note ] snapshot mode drift re-frozen` is harmless: Dropbox gave a frozen directory its write bit back and the driver re-froze it (2.6.8; before that it read `immutable path regained write permission` and every critic was re-sampled).
`FATAL: no working Codex CLI found; searched: …` *or* `FATAL: Codex executable is not executable: …`	The first: install or update the ChatGPT app, or `export CODEX=/path/to/codex`. The second names a path you set in `CODEX` or `I4R_CODEX_BIN` that does not work: unset both and the skill (2.6.10) picks the newest working CLI itself. Before 2.6.10 the second line was what every round printed after the ChatGPT app update of 2026-09-28 moved its CLI to `Contents/Resources/codex-cli/bin/codex`. No provider call was made.

`FATAL: model probe failed (permanent, unsupported_model): The 'gpt-6.1-sol' model is not supported when using Codex with a ChatGPT account … no critic calls were started`	The model probe, before any critic: nothing was spent. The ChatGPT app's Codex CLI is older than 0.159.0 (CLI 0.158.0-alpha.2.1 got exactly this on 2026-09-29), or the account does not offer Sol. Update the ChatGPT app and re-run the same round; if it still fails, the account does not offer Sol: sign the CLI in with one that does (the user's decision; for example an OpenAI API key, `printenv OPENAI_API_KEY | codex login --with-api-key`), check it with `python3 scripts/check_critic_access.py --force`, and re-run. Since 2.7.5 no audit moves off the shipped critic route.
`FATAL: this host cannot use the critic model gpt-6.1-sol … (from new_audit.sh, 2.7.5)`	Nothing was reserved or copied. The host's Codex account refuses Sol. Changing the account is the user's decision: sign the CLI in with an account that offers it (for example an OpenAI API key: `printenv OPENAI_API_KEY | codex login --with-api-key`) or use a host whose account does, then `python3 scripts/check_critic_access.py --force` and run `new_audit.sh` again. On a CLI older than 0.159.0 the message says to update the ChatGPT app first. Exit 4 instead (network, rate limit, model at capacity, server or timeout): just run it again; exit 5 (the CLI rejected the call itself, with no answer from the provider): update the ChatGPT app or the CLI.
`[fail ] critic_x permanent failure on attempt N (unsupported_model): The 'gpt-6.1-sol' model is not supported …`	A later round, or a rerun, from an account without Sol: the probe passed earlier and is cached per audit and CLI version, not per account, so the refusal comes at a critic. Each refusal cost that role one attempt, nothing else. Re-run the same round from an account that offers Sol (add `MAX_TRIES=<n>` if a role's attempts are used up); completed roles replay from cache. Since 2.7.5 no audit moves off the shipped critic route: finish from an account with Sol — do not start a new audit.
`Codex asks you to log in / quota exceeded`	Open the ChatGPT app, log in, confirm the workspace has credits, re-run the same round. Login and quota failures are never retried. The model probe runs once per audit and Codex CLI version: when it runs, it stops the round before any critic call; with the probe already cached, the first critic call fails instead, and each role spent at most one attempt.
`FATAL: new_audit.sh: no audit started, nothing reserved or copied: this skill copy is version … but the discipline tree's canonical skill is … (2.7.11)`	Nothing was reserved or copied. Reinstall from the shared zip (step 2) and run the same command. An older copy (2.7.10) reserves first and only `init_run.sh` refuses: then reinstall and rerun the same `init_run.sh` command on the same audit dir; do not rerun `new_audit.sh`.
`FATAL: data/… holds the package archive … and, beside it: … (from new_audit.sh, 2.7.11)`	Nothing was reserved. The paper folder holds an archive and loose author files. Ask the user which make the deposit, then rerun with `--loose include` (archive plus the files beside it, kept under `pkg/beside_archive/`) or `--loose exclude` (the archive alone).
`FATAL: … no audit started, nothing reserved or copied: pdftotext (Poppler) is not on PATH (2.7.11)`	Nothing was reserved. Install Poppler (the document build and the deposit checks need it too) and run the same command.
`status.py: critique round N stopped without its critic quorum (2.7.11)`	Every attempt of the listed roles failed (often a timeout or credits). Re-run the same `run_critique.sh` command on a 2.7.11 machine: completed roles replay from cache, and the run journals the pause's end before it dispatches.
`REFUSED: reserved by …`	Someone holds it. Pick another; check `reserve_paper.sh list`. Every refusal exits 1.
`RELEASE BLOCKED — fix the failures above and re-run`	`stage_release.sh`. Its own FAIL lines name the cause: a missing or symlinked mandatory input, more than one claims script, divergent finding projections, surviving verbatim author files, data files in the release, a participant code paired with a timestamp, or (check 3c, `check_deposit_values.py`) a value copied from the deposit's name, e-mail, birth-date, IP, address or phone columns: reword the finding by kind, file, column and count, never the value. A one-word name match only warns; exit 2 means a document could not be inspected.
`FATAL: document release: … N body page(s), cap 10`	The line lists *every* failed check, semicolon-separated. A comment is a paper: abstract ≤250 words on page 1, findings table in Appendix A. Let the pipeline fix and rebuild.
`FATAL: document release: document manifest builder code identity differs from the release verifier`	The document was built by an older skill: after the 2.6.9 upgrade every manifest from 2.6.8 or earlier is stale. Rebuild with `build_docx.sh <audit> <output-name>`, as the line says, then finalize, and move an old `release/` aside before restaging. The other one-time costs are in INSTALL.md, *Upgrading in-flight audits to 2.6.9*.
`FAIL: appendix finding IDs must occur exactly once … / rows never tagged in the body`	`check_consistency.sh` emits seven distinct join failures. Mid-round, with draft P-rows still in the registry, the fix is `project_draft_claim_tags.py`.

`FINALIZE FAILED — critique routes/results are not bound to immutable runtime projections`	`validate_round_projections.py`, which finalize runs first. A round's recorded inputs no longer match its snapshot.
`NOTE: compliance_pass is false`	The audit filed anyway. Usually a phase never closed or no `delivery` event — see section 5. It must be stated in the release notes.
`R notebook exits nonzero`	**A claim genuinely failed to reproduce — that is a finding, not a bug.** Read `verification_summary.csv`. Never "fix" the check to pass.
`An empty directory reappears after cleanup`	Dropbox resurrecting a deleted directory. Documented in INSTALL.md; leftover `.finalize-tx.*` dirs are this, not a crash.

10 What it costs, and what you are responsible for

Pipeline, recorded	**72.8M tokens** for the one audit with complete token accounting (98% of it cache reads; fresh input+output is ~3.2M). Two others record 149M and 6.4M but are missing 9% and 43% of their calls — treat those as floors.
Codex side	6.4M–27.8M tokens across four runs. The smaller half of the bill, and **not** stable.
The session you drive it from	**5–9× the recorded figure**, and it appears in no artifact. End to end an attended audit has measured 450M–1.4B tokens. An unattended run avoids nearly all of it.
Where it goes	Round 0 fan-out is 66–81% and you do not control it. Critique is ~20%; the rounds past the target of 2 are ~9%. Capping rounds saves ~9% and costs findings.
Human time	Recorded on two legacy audits at **145 person-minutes** each. The modern three record `null`. If nobody timed it, say so rather than leaving it blank: `log_event.sh human_effort notes='unattended; no human performed any phase'` — only `notes=` earns the softer finalize wording.

Interject freely. A `/goal` session answers questions mid-run without derailing the pipeline. **Read the adjudications** — when the coordinator re-runs a disputed computation it says so, and that is the audit's core.

Critics finding problems with the audit itself is the design working. Across the calibration audits most blocking criticisms were defects in our own draft. An audit whose critique rounds find nothing is the suspicious one.

Do not edit the comment by hand. The gates, records and claim numbering assume machine-produced text; a hand edit breaks the joins and skips the checks. Tell the session what looks wrong and let it regenerate.

You are not the referee. Every report goes to a replicator who verifies each claim manually, and the coordinator reads every comment before submission. Your job is to produce it cleanly.

Skill version **2.6.2**. Counts, board state and cost figures were measured against this tree on 2026-09-08 and will drift — re-derive before quoting. Where this sheet and INSTALL.md disagree, INSTALL.md is the maintained reference and this sheet is the one that is stale.